

National University of Computer & Emerging Sciences
Karachi Campus



GUI Performance Tracker and Predictor

Project Proposal
Programming Fundamentals
Section: 1-G

Group Members:
25k-0980 Safee Imran
25k-0722 Hasan Khan

Project Proposal

Introduction

1st section:

An attendance tracking system that tracks the users attendance in real time. Along with that, it has a calculator that outputs the number of absences one has for each course.

Also on the basis of two ML models we calculate the probability of a person completing 80 percent attendance.

2nd section:

A grades tracking system that tracks the existing grades of a student and displays it along with the attendance. This section also includes 2 ML models that judge the possible performance of the student at the end of the semester. The performance will be displayed in the form of an expected GPA at the end of the semester.

Problem Statement

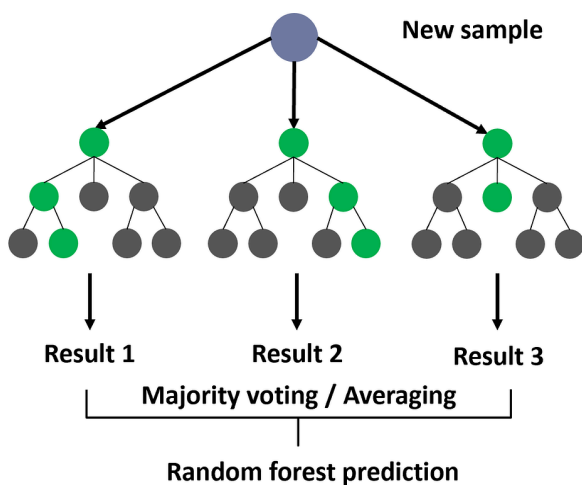
A way to predict the performance and attendance (expected at the end of the semester or year based off of existing data).

Proposed Solution

Setting up the Framework:

For the fundamental framework of the project we'll build an attendance tracking system using matrices in C for each course. We'll likewise implement the same system for tracking the grades of a student in each course.

Random Forest (Primary Algorithm for this project):



Random Forest builds a large collection of trees, each trained on a random subset of the dataset using bootstrap sampling. At every split within each tree, only a random subset of features is considered, which reduces correlation between trees and increases diversity.

For classification tasks, each tree in the forest casts a "vote" for the predicted class. The final output of the Random Forest Classifier is the class chosen by the majority of trees (majority voting). This method significantly reduces variance compared to individual decision trees, while still maintaining low bias.

Probability and Statistics in vanilla C:

Math (Splitting Criteria):

1. Gini Impurity (for classification):

Gini Impurity is a rather complex phenomenon that evaluates the class probability of a student exceeding the (80%) attendance benchmark on each course.

$$Gini(S) = 1 - \sum_{i=1}^k p_i^2$$

This formula is based on class probability not binary probability.

Regression analysis:

Regression Analysis is for evaluating the possible performance of a student in terms of expected CGPA at the end of the semester or year. This part of the project takes two ML models into account.

1. Variance reduction/MSE(mean squared error):

Using the MSE method we reduce the weighted variance in the Random Forest Classifier.

$$Var(S) = \frac{1}{|S|} \sum_{i=1}^{|S|} (y_i - \bar{y})^2$$

Here $\bar{y}$ is the mean of the target value “S”.

2. Regression (Performance):

For the final prediction of the performance of a student's expected end of session CGPA. We use the following formula to present the final calculated CGPA of “t” decision trees.

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T h_t(x)$$

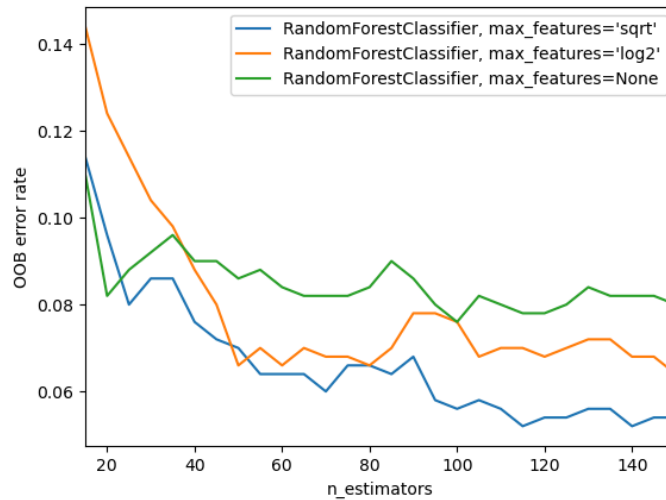
Where $h_t(x)$ is the prediction of the “t-th” tree in the Random Forest Classifier.

Testing of the ML models used:

OOB (Out of Bag) error estimation:

The OOB samples provide a built-in mechanism for evaluating model performance without requiring a separate validation set. After training, each OOB sample can be passed through the

subset of trees that did not use it for training. The aggregated prediction from these trees is then compared with the true label of the sample. By repeating this process for all OOB samples, the algorithm computes the **OOB error rate**, which serves as an unbiased estimate of the model's generalization error.



Reference OOB error rate chart

OVERVIEW:

A model with a possible accuracy of 85% or above that predicts the overall performance of the student integrated in a GUI based system that also stores the records for all the existing and predictable data easily accessible by the user.

- **Tools & Technologies**

Vanilla C (no external libraries included) , OpenGL for GUI